专业：信息工程
姓名：
学号：
日期：2026.5.12
地点：紫金港东 4

浙江大学实验报告

课程名称：通信原理实验　　指导老师：金向东、龚淑君　　成绩：
实验名称：熟悉 Labview　　实验类型：综合型实验　　同组学生姓名：

一、实验目的和要求

(1) 熟悉 LabVIEW 通信设计套件
(2) 会使用 While/For 循环处理数据
(3) 熟悉多种数据类型
(4) 会使用数组，访问和处理元素

二、实验设备

安装 LabVIEW 环境的电脑 1 台

三、实验概要

本实验课程的目的是在通用软件无线电外设（USRP）上应用、实践关于软件无线电的数字通信理论。USRP 结合了 LabVIEW 软件，该软件是一个使用模块和连线的图形编程软件。因此有必要熟悉 LabVIEW 语言并能写一些代码。

四、实验内容和步骤

1. 建立项目和熟悉界面

1)运行电脑中的软件：NI LabVIEW NXG 2.1；
2)新建一个项目:点击文件/新建/项目；
3)在项目中添加一个新的 VI：点击文件/新建/VI；
4)将项目重命名为 Lab1：在窗口左上角右击未命名项目并点击重命名；
5)每一个 VI 有两个窗口：前面板和程序框图。前面板是用户和 VI 进行交互的地方，它包括很多控制输入和显示输出参数端口。程序框图窗口包含从控制输入端接收输入数据，通过连接模块进行处理，并且通过显示输出进行显示的代码（流程图）。

2. while 循环

While 循环是一个控制流语句，用于重复执行一个函数，直到满足给定的条件终端。条件终端定义循环结束的时间。条件终端有两种设置：如果为 True 则继续和如果为 True 则停止。如果设置为 True 继续，则仅当布尔控件返回 True 时才进行循环。如果设置为 True 则停止，布尔值为 True 时，循环将中止。默认情况下，条件终端是“True 则停止”。

例程 1：使用 while 循环，产生一组 0 到 10 之间的随机整数，当产生的数与用户给定的数值匹配是，停止产生随机数

1) 新建一个 VI
2) 在程序框图窗口放置一个 while 循环（左侧程序流模块栏里选取）
3) 放置随机数模块产生随机数（数据类型/数值/随机数），产生 0 到 1 之间的随机浮点数。
4) 为了产生 0 到 10 之间的随机数，放置乘法模块和数值常量输入控件以及最近数取整模块（数据类型/数值）。
5) 创建一个数值显示输出端口，显示每一次循环产生的随机数：放置一个数值接线端（数据类型/数值），这个接线端默认的行为特性是“控制”（从前面板输入），需要改变其特性为“显示”（输出到前面板），重命名显示端口为 Random Integer。
6) 创建一个数值输入控制端口（数值接线端），接收匹配的整数。重命名控制端为 Number to match。
7) 使用“等于？”模块（数学/比较），比较两个数值的大小。
8) 在前面板窗口创建一个停止按钮（按钮/停止按钮），回到程序框图窗口，在未放置项将停止控件拖放在框图中。
9) 放置“或”模块（数据类型/布尔），将布尔控制两个输入分别与比较器的输出、停止按钮连接。当输出随机数与输入匹配或者按下停止按钮，循环将中止。
10) 回到前面板，将未放置项拖放进窗口界面。
11) 在前面板输入一个匹配数值，运行程序，观察结果。
12) 在程序框图窗口上方菜单条中，点击“高亮显示执行结果”，运行程序的时候，可以观察到程序框图中数据流的变化。

3. 数据类型（簇）

在 LabVIEW 中，每一个对象和连线都跟数据类型有关系。LabVIEW 支持很多数据类型，它们由不同的颜色、形状区分。部分数据类型如图 1 所示：



图 1　部分数据类型

例程 2：接收一串字符串，将其反转；对给定的数组元素数值增加偏置量。

1) 新建一个 VI
2) 在前面板，创建一个空白簇。
3) 在空白簇中放置一个字符串输入控件、一个数值输入控件和一个数值数组输入，将数组尺寸改为 5（可通过鼠标拖拉数组外框或直接在属性里修改）。在簇外面再放置一个字

符串输入控件和一个数值数组输入，将它们改为显示模式。

4) 在程序框图窗口将未放置项放置在框图界面里，在程序框图中放置簇属性模块（数据类型/簇），以便从簇中读取各个数值。将簇控件与簇属性模块相连，鼠标往下拖拉簇属性模块外框显示更多的属性端口。
5) 使用反转字符串函数（数据类型/字符串）反转字符串，使用“加”函数（数学/数值）将偏置与数组相加。
6) 在前面板输入控制值，运行程序，通过显示输出确认结果是否正确。
7) 将改变的值再结合放入簇：复制、黏贴前面板中的簇并将它的属性改为显示。
8) 将新的簇（未放置项）放置在程序框图中，为了将改变的字符串、数组和新的簇结合在一起，需要再次用到簇属性模块。因为要将数据写入簇，所以需要将簇属性设置为“全部为写入”。
9) 运行程序并比较结果。

4. 数组

数值中的元素都是相同类型的数据，可以是数值、布尔、路径、字符串、波形和簇。当处理一批类似数据或作重复计算时，可以考虑使用数组。存储波形数据或循环中产生的数据时，使用数组是一个理想选择。

例程 3：给定一个数值数组，找出其中的最大元素。

1) 新建一个空白 VI
2) 在前面板添加一个数值数组输入，通过往下拖拉数组外框将元素数量增加到 5。将数值数组输入重命名为“Array”
3) 添加一个数值输入控件，将它改为显示模式并重命名为“Maximum value”。
4) 在前面板添加一个 For 循环，将“Array”终端放在循环外面，“Maximum value”终端放在循环里面。
5) 将数值终端与 For 循环的左边框相连，在边框上会产生一个“隧道”，隧道使得循环与外面的模块能够进行数据通信。隧道有索引使能功能，当它设置为自动索引使能时，输入循环结构的数组元素在每一次循环的时候送入一个元素。如果索引没有使能，循环结构将一次调用数组中的全部元素。
6) 为了在 For 循环中对输入数组的元素逐个跟设定的最大值进行比较，需要在 For 循环中添加一个“大于？”函数（数学/比较）。再放入一个“选择”模块（程序流）。
7) 假如当前输入循环的数组元素比最大值大，需要更新最大值。将隧道输出与“大于？”模块的 X 端相连，同时连接到“选择”模块的真值输入端。将“大于？”模块的输出与“选择”模块的选择端连接。
8) 我们需要存储当前的最大值以备下次循环迭代的时候使用。移位寄存器是将两次相邻迭代联系起来的数据传输通道。将“选择”模块的输出连接到循环的右侧边框线上，会默认建立一个“隧道”，点击鼠标右键，将它改为移位寄存器模式，这时鼠标光标也变成移位寄存器的图标，点击循环的左边框，完成移位寄存器的放置。（移位寄存器的左侧方框可以看作是来自上一次迭代结果的输入，右侧方框是本次迭代的输出）
9) 移位寄存器的左侧方框是上一次迭代产生的最大值，用来与当前输入的数组元素进行比

较。将移位寄存器的左侧方框与“大于？”模块的 Y 端以及“选择”模块的假值连接。

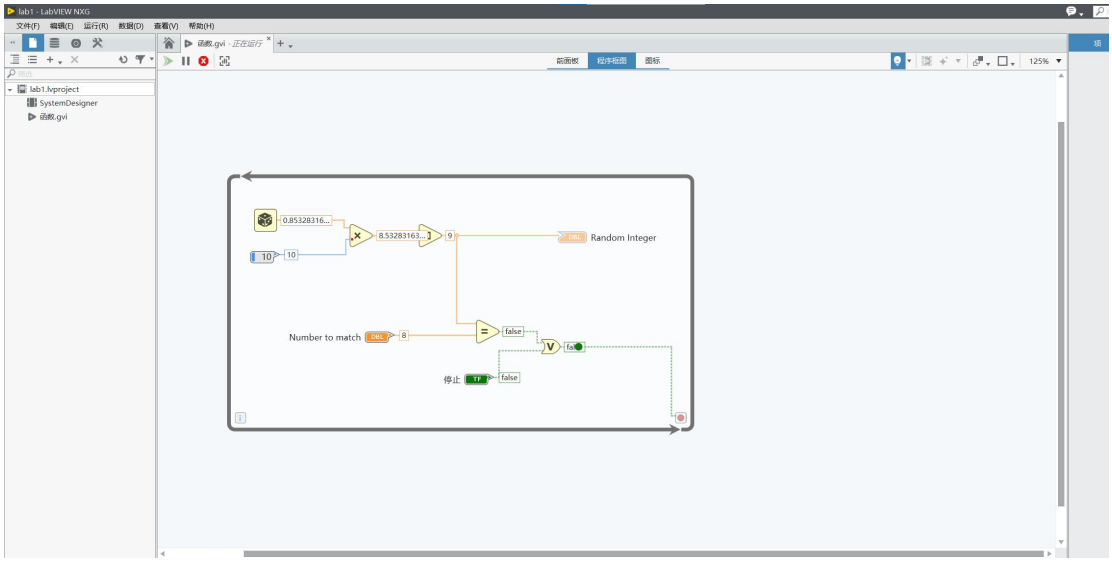
10) 将“选择”模块的输出与 Maximum value 显示控件相连。

11) 在前面板给 array 输入数值，运行程序确认是否正确。

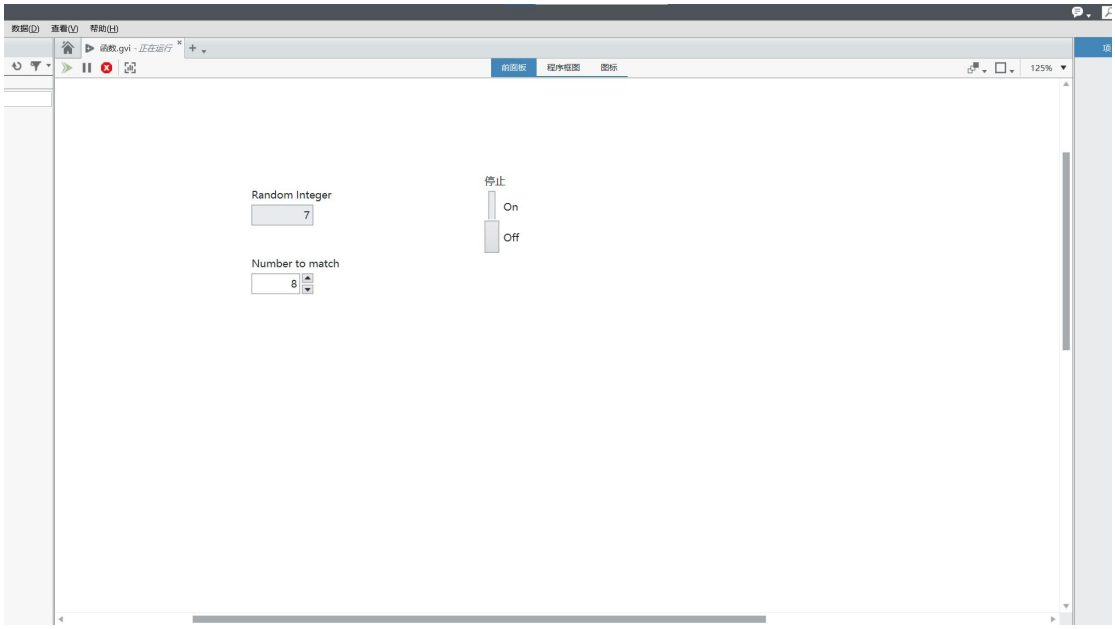
五、示例完成以及问题回答

1. 问题 1

1) 完成以上示例，在实验报告中提供程序框图和前面板运行结果



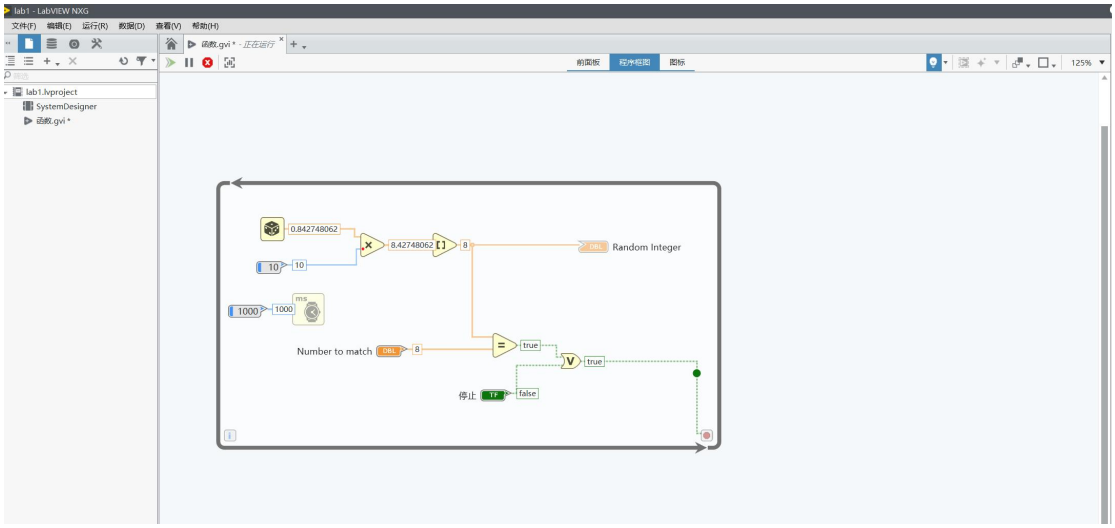
程序框图



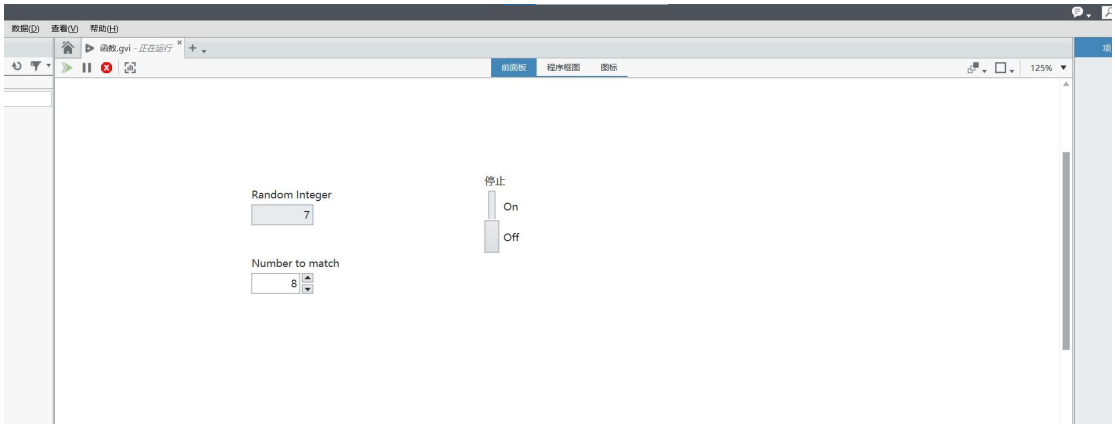
前面板（图示产生的随机数为 7，不匹配，因此程序继续）

2) 给 while 循环增加一些延时（比如 1 秒钟），以便能观察到数据的变化

增加延时后，前面板数据刷新变慢，易于观察

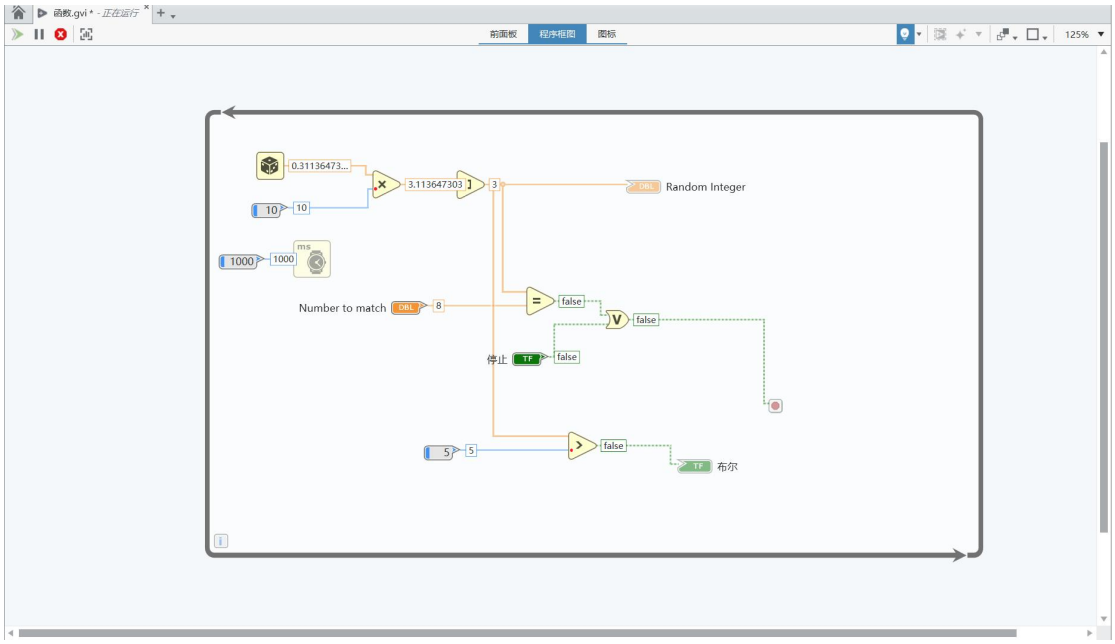


程序框图

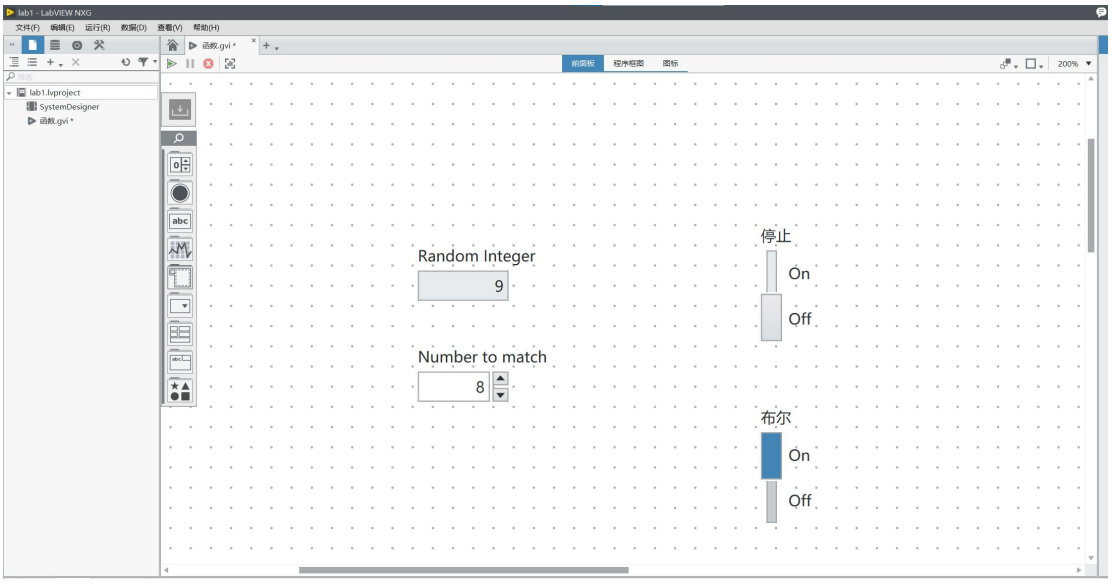


前面板（增加延时后刷新变慢）

3) 增加一个布尔指示器，当随机数大于 5 时打开，小于或等于 5 时关闭



程序框图

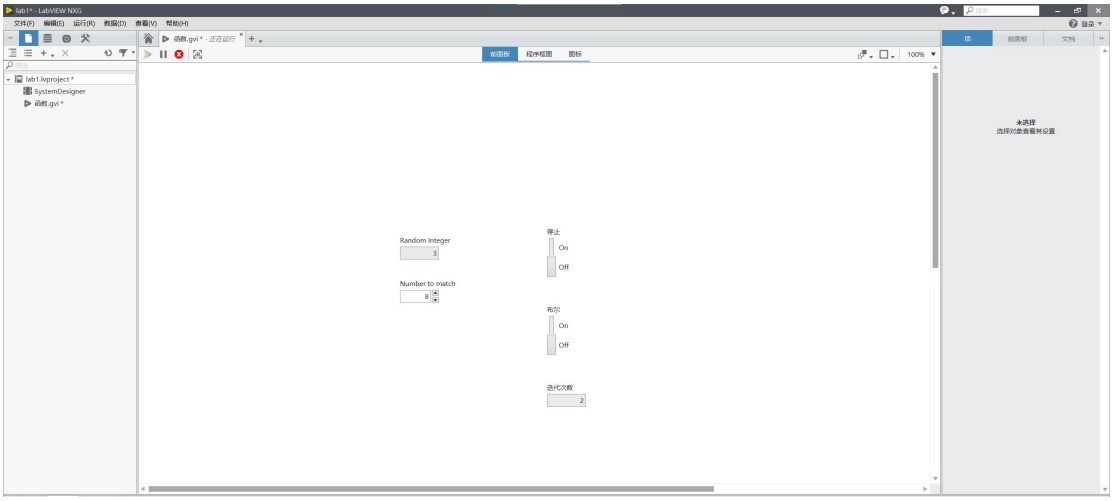


前面板（产生的随机数此时为 9，大于 5，判断为 True，布尔开关 On）

4) 增加一个输出端显示循环的次数



程序框图（while 循环框右下角会记录迭代次数，因此插入一个显示模块即可）

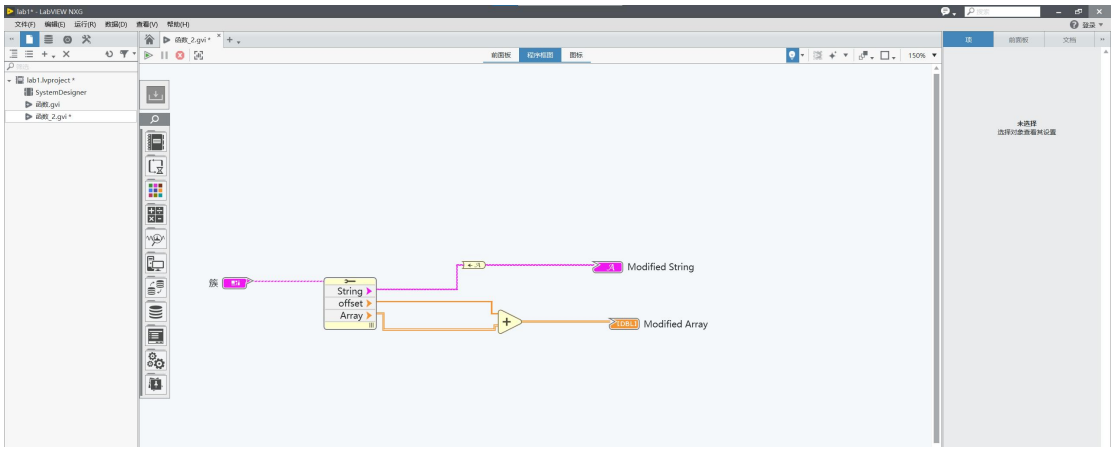


前面板

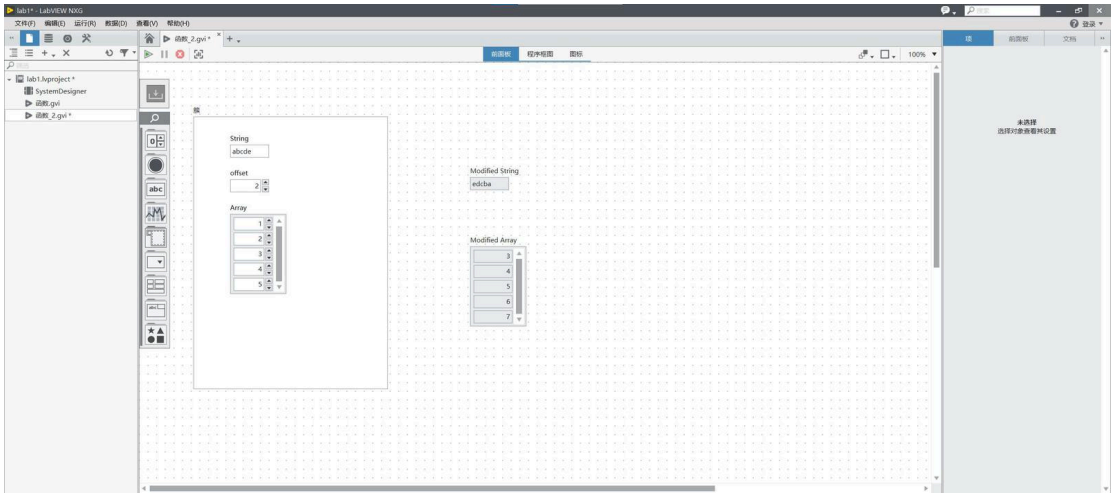
2. 问题 2

1) 完成上面的示例，在实验报告中提供程序框图和前面板运行结果。

a) 反转字符串和“加”函数

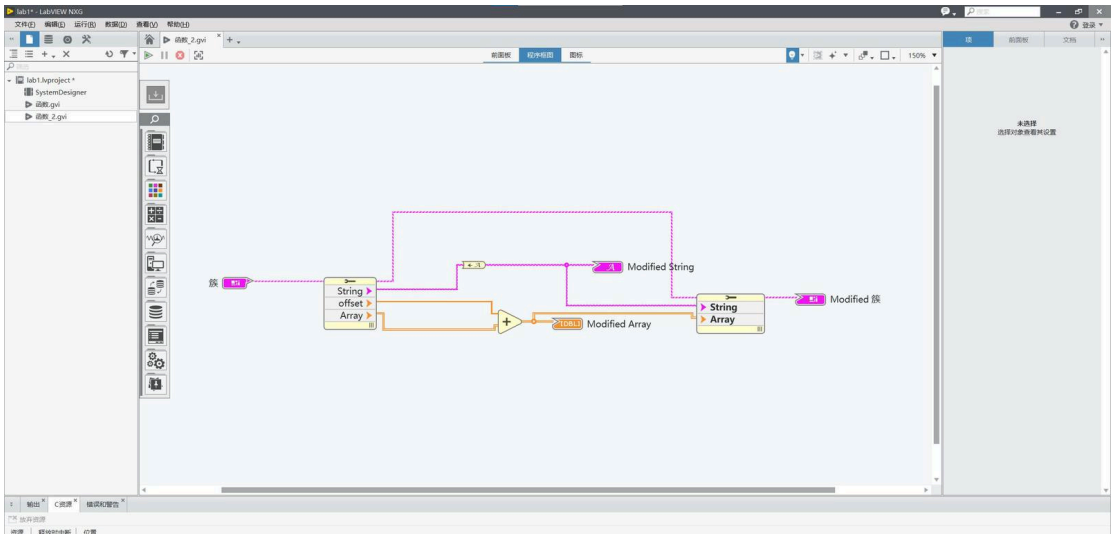


程序框图

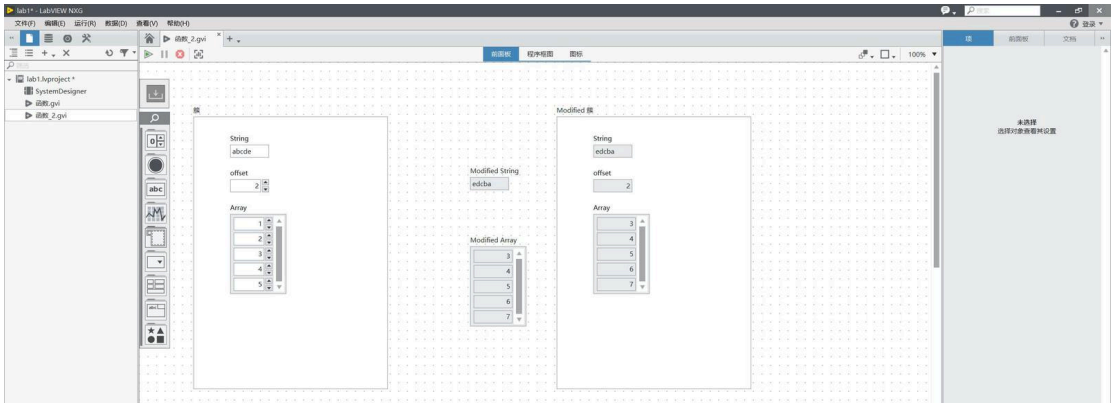


前面板（可以看到原字符串 abcde 反转后变为 edcba，数组值整体增加 offset=2）

b) 将改变的值再结合放入簇，将新的簇放置在程序框图中，写入簇



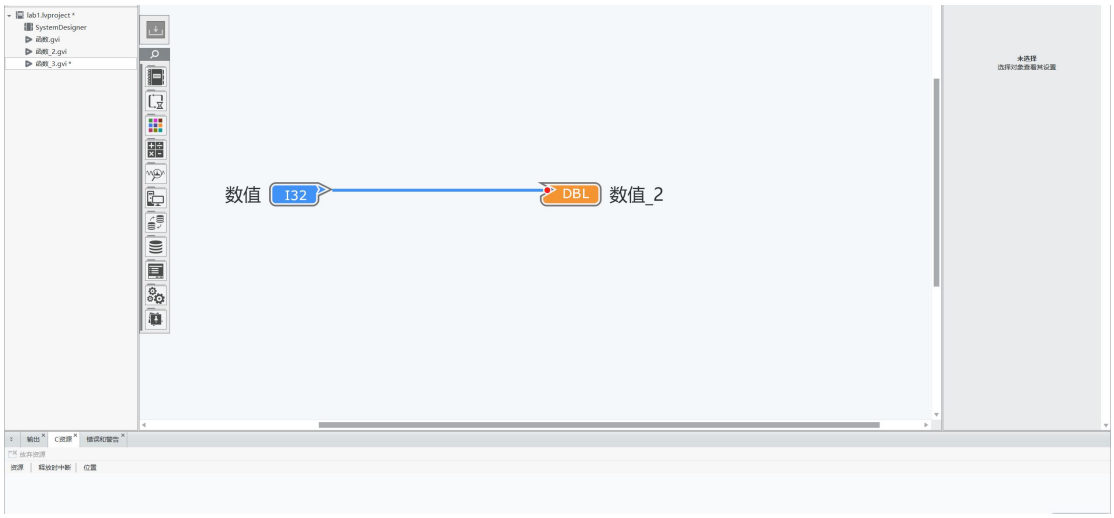
程序框图



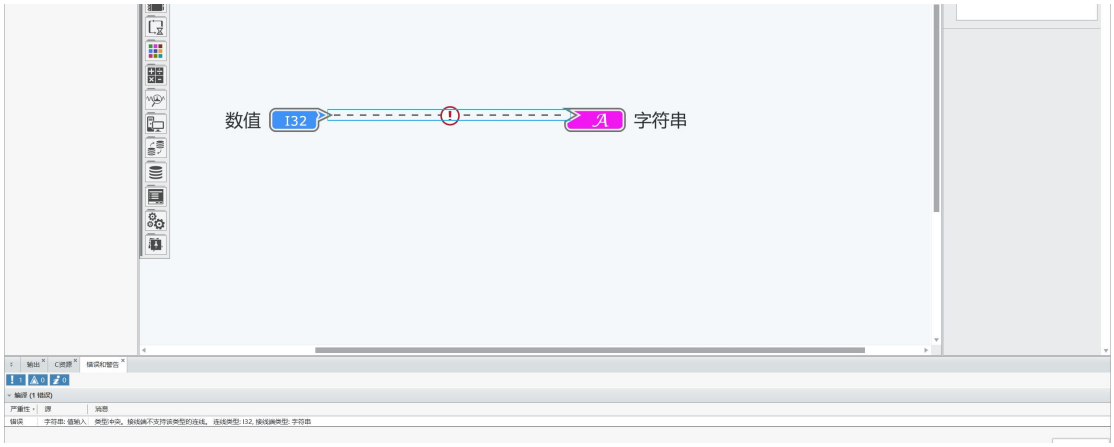
前面板（输出字符串与输入反序，输出数组由输入数组值加上偏移量得到，偏移量显示不变，结果正确）

2) 是否能将一个整型控制输入与双精度类型显示输出相连？整型输入能否与字符串输出相连？如果不可以，需要使用什么模块进行类型转换？

a) 整型控制输入与双精度类型显示输出可以直接相连

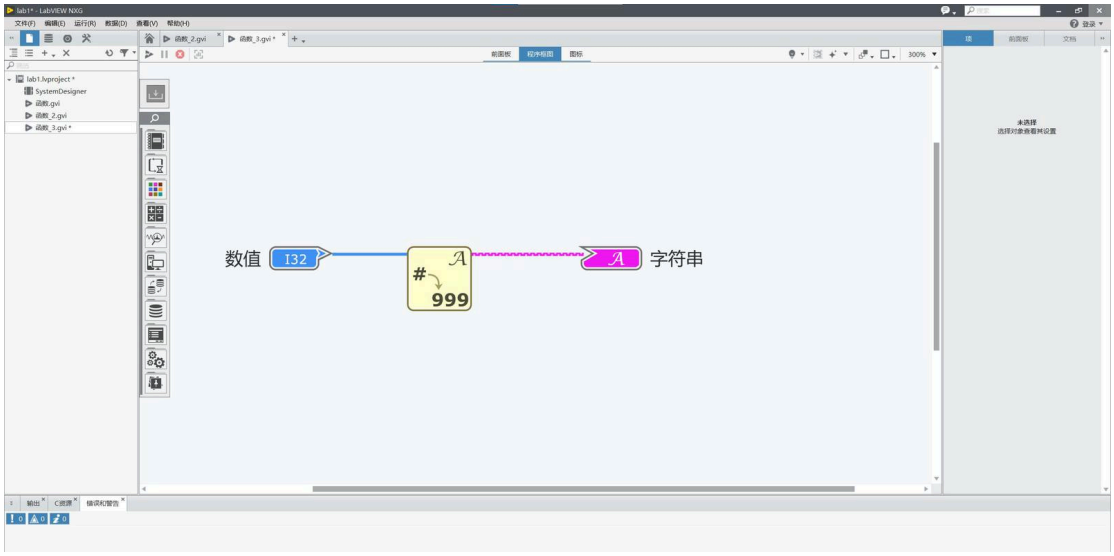


b) 整型控制输入与字符串显示输出不能直接相连，会出现报错，因为 LabVIEW 遵循严格的数据类型匹配原则，强行连接会导致连线断裂并提示数据类型不兼容。

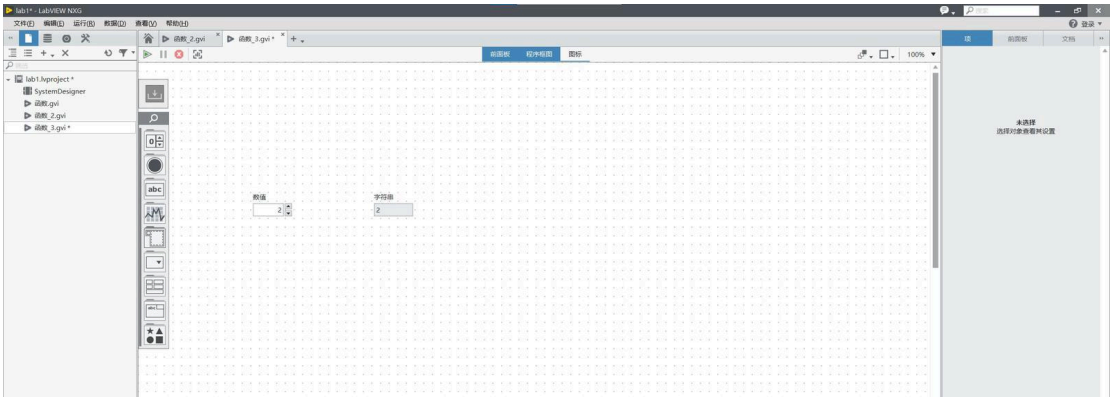


c) 将整型输入转换为字符串输入需要添加“数值十进制数字符串”控件





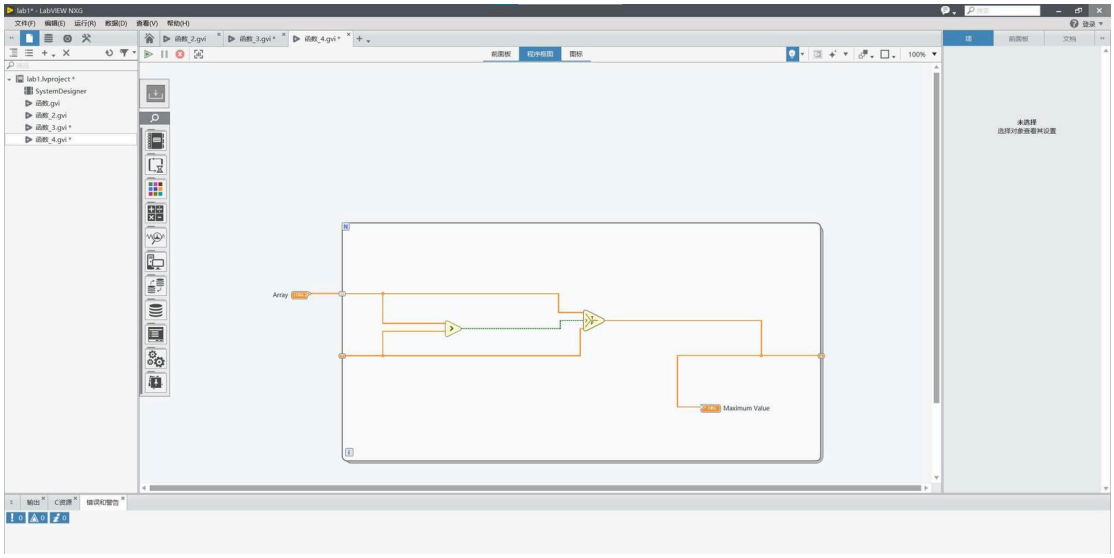
程序框图（添加控件后不再报错）



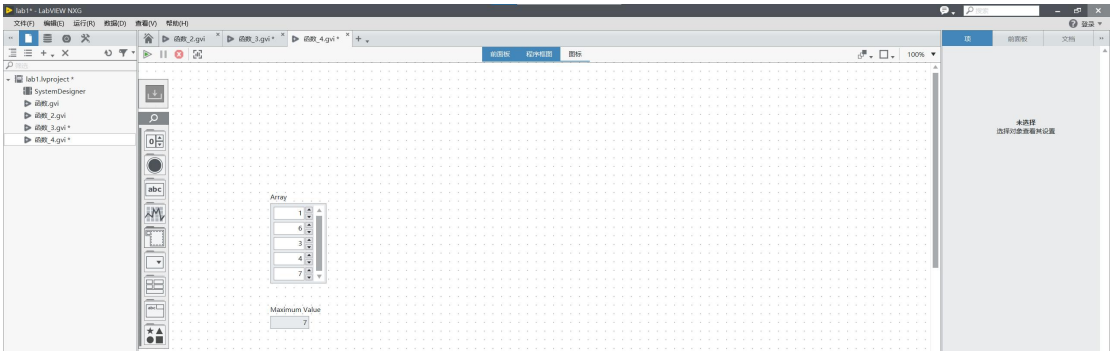
前面板

3. 问题 3

1) 完成上面的示例，在实验报告中提供程序框图和前面板运行结果



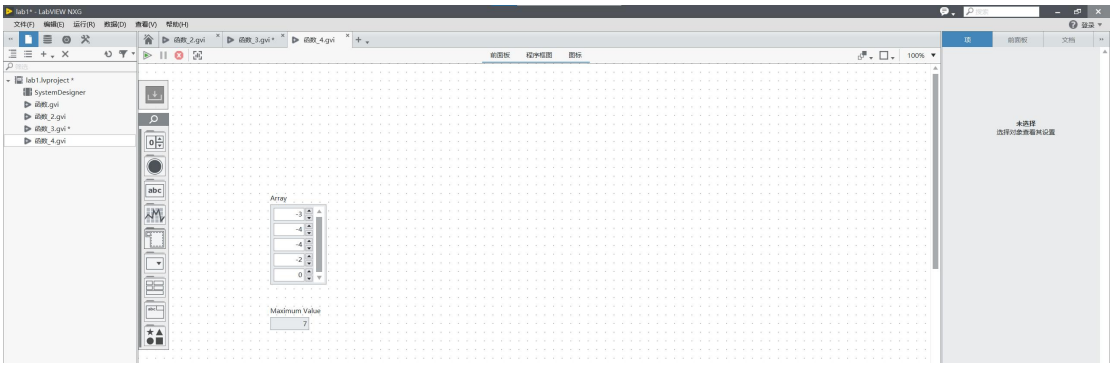
程序框图



前面板（执行完后顺利找到数组最大值 7）

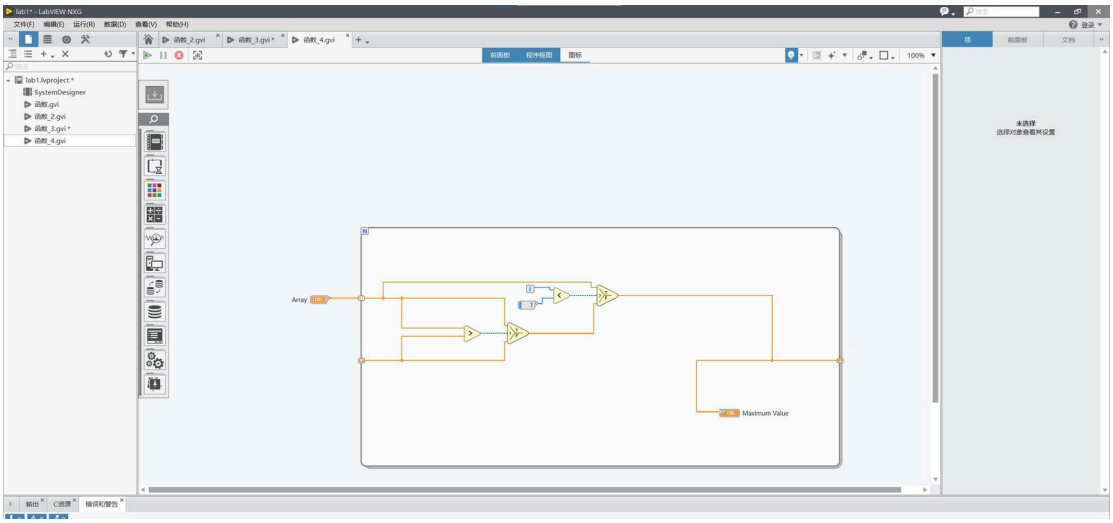
2) 将数组元素最大值改为 0, 再次运行程序, 还能找到最大值吗？更改例程, 使得每运行 一次程序都能找到最大值

将数组元素最大值改为 0 后，再次运行不能找到正确的最大值 0，而是得到 7

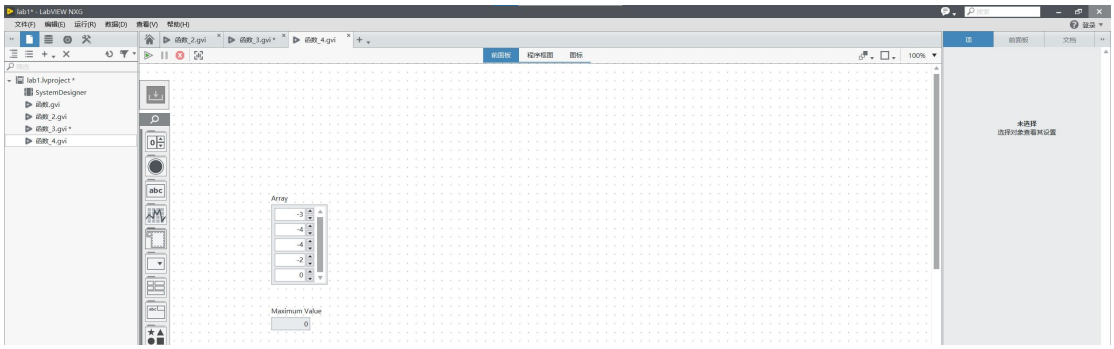


前面板

当数组元素最大值改为 0 后，程序无法找到正确的最大值。根本原因在于移位寄存器未进行初始化操作，导致其保留了同一内存地址中上一次程序运行的残留状态（即历史最大值 7）。在当前循环中，由于输入数组的所有元素（皆小于或等于 0）均小于该历史值 7，条件判断始终为假，导致错误的历史值被持续传递输出。在程序框图中加入对首次迭代 (i=0) 的判断，强制将数组首元素作为初始基准值写入寄存器，可彻底解决此问题。



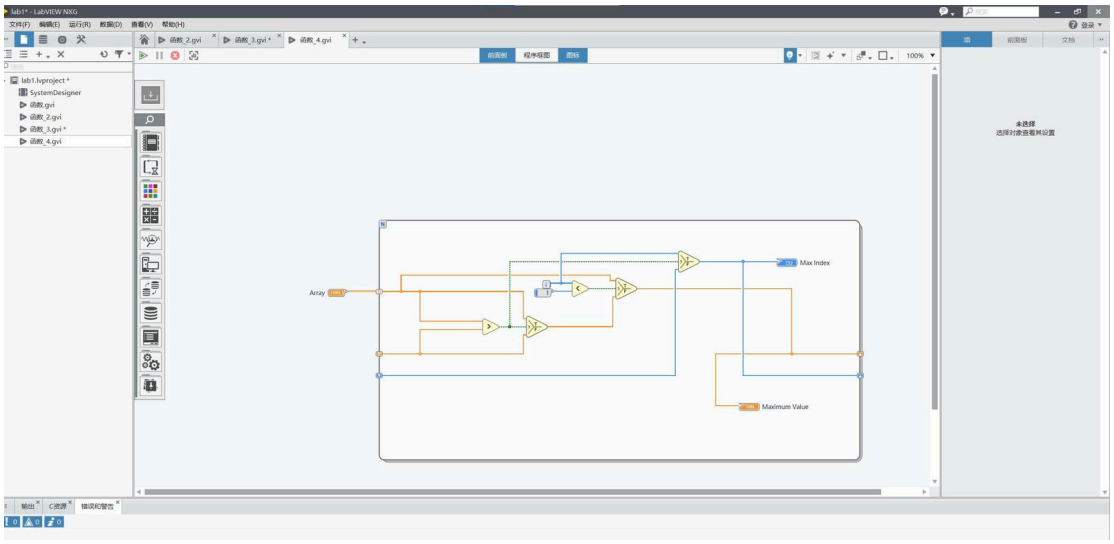
程序框图 (加入一个判断, 如果迭代处于第一轮, 则为 F, 将数组第一个值赋入 Maximum Value，当迭代进入第二轮以后进行比较)



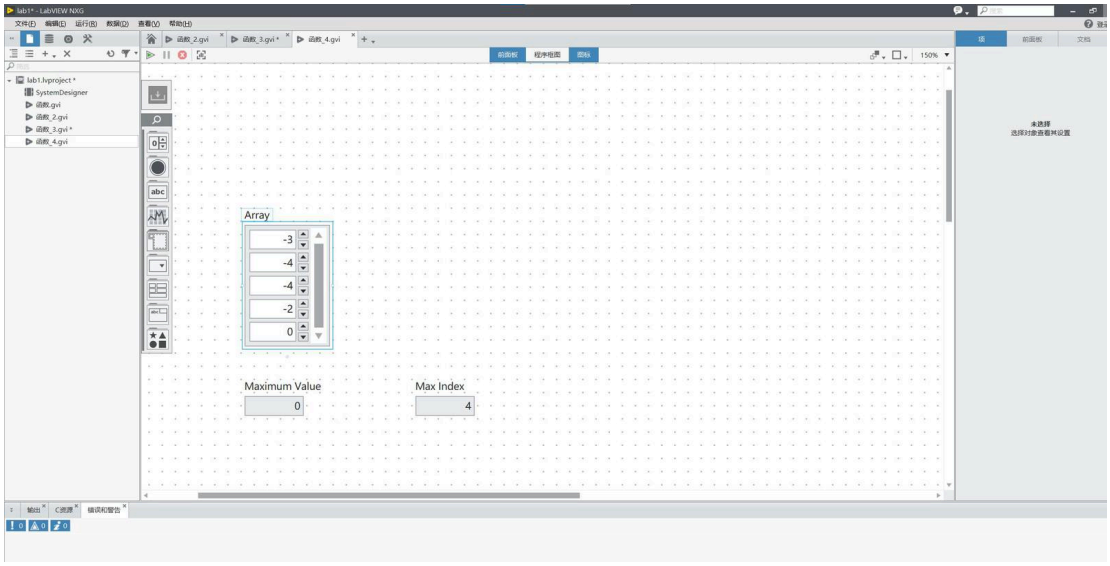
前面板（得到最大值 0）

3) 在问题 2 的基础上，更改程序框图，能够得到最大值的索引值（在数组中的位置）

在数组值进行比较时，同时记录此时的 Maximum Value 和对应的 Index，为了避免问题 2 出现的情况，第一轮迭代时需要对 Maximum Value 和对应的 Index 赋入初值



程序框图（同时记录 Value 和 Index）



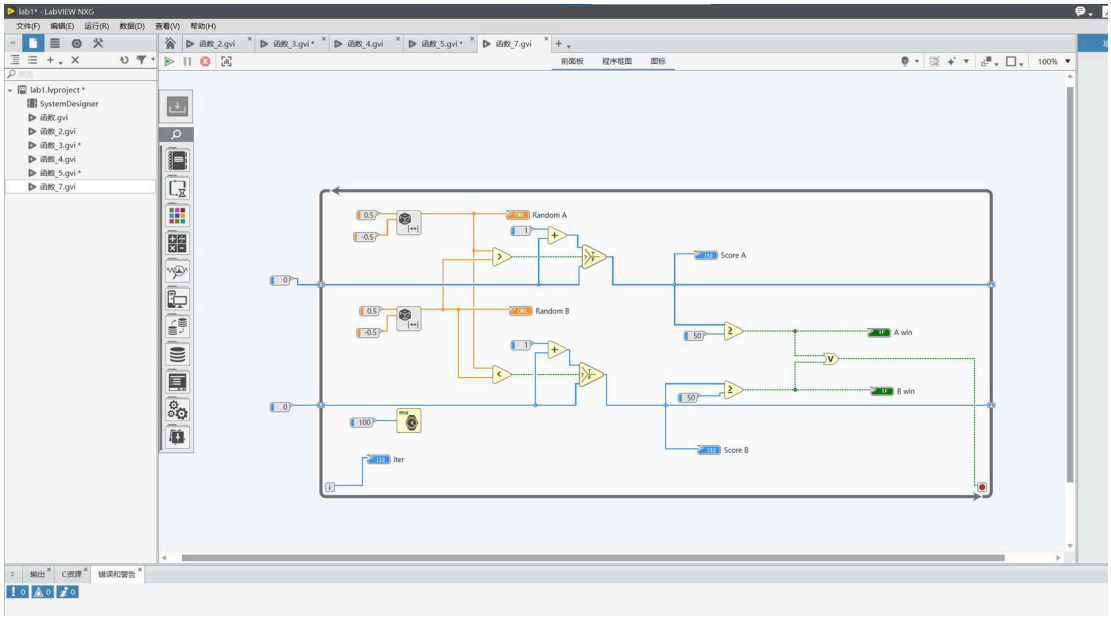
前面板（得到最大值为 0，数组索引值为 4，即第五个数字）

4) 完成以下 VI：产生两个数值范围在-0.5 到 0.5 之间的随机数，比较这两个随机数值的大 小，如果第一个数比第二个数大，Team A 得分；如果第二个数比第一个数大，Team B 得分。当其中一个队得分达到 50 时，程序停止运行。显示最终的两个随机数、循环的 次数、Team A 和 Team B 的得分以及哪一队赢。

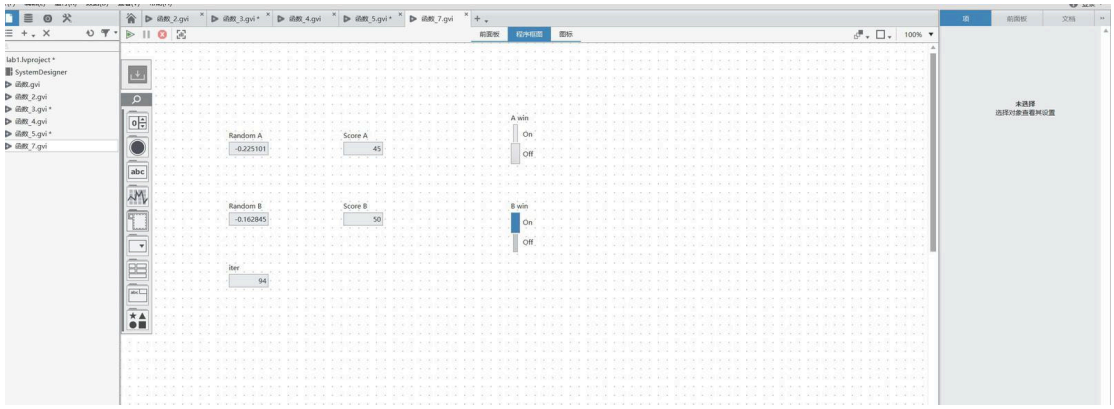
a) 初始化： 程序启动前，通过 While 循环外部的常量接线端为两个移位寄存器赋初值 0，用于清零两队的初始得分。

b) 循环迭代逻辑： 每次迭代生成两个 [-0.5, 0.5] 范围内的双精度浮点随机数并进行比较。若 A>B，则通过移位寄存器和选择器使 Team A 累加 1 分；反之则 Team B 累加 1 分。程序配置了 100ms 的等待函数以控制刷新率，并实时更新得分与迭代次数。

c) 停止与判决条件： 将两队的实时得分分别与常量 50 进行“大于等于”比较。当任一队伍得分达标时，产生的布尔真值将通过“或”门触发条件接线端（True 则停止），终止循环并同时点亮对应获胜方的布尔指示灯。



程序框图



前面板